

快速开始

这里说一下如何快速在板子上复现本组最终作品

1. 切换到源码目录 `cd rou/yolo`
2. 此目录结构说明

当前目录

```
├─ __pycache__/
├─ data/                # 测试视频
├─ model/
├─ runs/                # 预测结构保存位置
├─ bestmax.om           # 最终使用模型
└─ om_video_track_smooth.py  # 预测python代码
```

3. 直接推理 `python ./om_video_track_smooth.py`

这个脚本是可以直接使用的，而且目前的测试视频也很适用，10多秒不会花很多时间，也可以在此代码配置部分修改测试视频

```
# --- 配置部分 ---
OM_MODEL_PATH = "bestmax.om" # OM 模型文件路径
VIDEO_PATH = "data/test2.mp4" # 输入视频文件路径
OUTPUT_VIDEO = "runs/test2_om.mp4" # 输出视频文件路径
DEVICE_ID = 0 # 计算设备 ID (NPU)
CLASS_NAMES = ['kayak', 'fishing boat'] # 类别名称列表
```

压缩包说明

说明一下拿到这个压缩包，文件情况

当前目录

```
├─ datasets/            # 数据集
├─ yolov12_ultimate/    # 训练结果
├─ om_video_track_smooth.py  # 板子上的预测视频
├─ README.md
└─ train_yolov12.py      # 训练脚本
```

工程全流程

环境概述

硬件

- **Windows**: 加速训练
- 虚拟机: ATC模型转换
- **Atlas 200I DK A2**: 推理运行

一、Windows训练环境搭建

这个由于做过yolov5的训练环境搭建, yolov12和他差不多, 不做过多说明

二、数据集与训练

1. 数据集

- 数据集目录结构

```
datasets/
├─ train/
│   ├─ images/
│   └─ labels/
├─ valid/
│   ├─ images/
│   └─ labels/
├─ test/
│   ├─ images/
│   └─ labels/
└─ data.yaml
```

TXT

- data.yaml

```
1  # 数据集路径
2  train: ../datasets/train/images
3  val: ../datasets/valid/images
4  test: ../datasets/test/images
5
6  # 数据集类别
7  nc: 2
```

2. 训练

训练脚本在压缩包

三、ATC模型转换

1. 平台选择

注意

- 开发者套件做转换时，内存太小 容易卡死报错
- 用虚拟机转换，且推荐分配12G内存

2. 环境搭建

- CANN SDK

```
1 | cd /home/user/Downloads
2 | chmod +x Ascend-cann-toolkit_*.run
3 | sudo ./Ascend-cann-toolkit_*.run
```

- 环境变量

```
1 | export PATH=/usr/local/Ascend/ascend-toolkit/latest/atc/bin:$PATH
2 | export LD_LIBRARY_PATH=/usr/local/Ascend/ascend-
  toolkit/latest/fwkacclib/lib64:/usr/local/Ascend/ascend-
  toolkit/latest/atc/lib64:$LD_LIBRARY_PATH
3 | export PYTHONPATH=/usr/local/Ascend/ascend-toolkit/latest/atc/python/site-
  packages:$PYTHONPATH
4 | export ASCEND_OPP_PATH=/usr/local/Ascend/ascend-toolkit/latest/opp
5 |
6 | source ~/.bashrc
7 |
8 | atc --help
```

3. 转换

参数需与模型适配，可用netron查看参数

```
1  atc --model=best.onnx \  
2      --framework=5 \  
3      --output=runs/best_960_fp16 \  
4      --input_format=NCHW \  
5      --input_shape="images:1,3,960,960" \  
6      --soc_version=Ascend310B4 \  
7      --precision_mode=force_fp16 \  
8      --log=error
```

四、推理预测

预测脚本在压缩包

五、模型优化方向

1. 目标框“呼吸效应”与抖动 (Breathing Effect)

- 问题现象：检测框的大小和位置在连续帧之间剧烈波动，看起来像在“呼吸”或快速闪烁，不够平滑。
- 原因分析：原始检测模型的输出本身存在逐帧噪声，直接画出来会导致视觉上的抖动。
- 解决方案：
 - 引入了卡尔曼滤波 (Kalman Filter)：在 KalmanBoxTracker 类中实现。它不仅利用当前帧的检测结果，还结合了上一帧的预测状态（位置+速度），对框的位置进行平滑估计。
 - 实现了移动平均 (Moving Average)：在 Tracker 中维护 box_hist 队列，取最近 TRACK_BUFFER_SIZE (12帧) 的平均值作为最终显示框，进一步消除高频抖动。

2. 重复检测框 (Duplicate Boxes)

- 问题现象：同一个目标上出现多个重叠的框，或者一个目标被识别为多个 ID。
- 原因分析：YOLO 模型可能对同一目标输出多个预测，且默认的 NMS（非极大值抑制）阈值不够严格；或者跟踪器在匹配失败时错误地创建了新轨迹。
- 解决方案：
 - 激进的 NMS 阈值：将 IOU_THRES (NMS阈值) 从默认值降低到 0.2。这意味着只要两个框重叠超过 20%，就只保留置信度高的那个，强力去重。
 - 严格的跟踪匹配：降低 TRACK_IOU_THRES 到 0.15，使得检测框更容易匹配到已有轨迹，避免因微小偏差而判定为“新目标”。
 - 二次去重逻辑：在绘图前增加了一段额外的逻辑 (Line 941-968)，对比所有待绘制的框，如果发现不同 ID 的框重叠严重，强制隐藏 ID 较新或置信度较低的那个。

3. 框位置偏移 (Box Offset)

- 问题现象：检测框虽然能跟住目标，但位置总是有整体偏移（例如飘在目标上方），或者长宽比不对。
- 原因分析：图像预处理时直接 Resize 导致了拉伸变形，或者后处理映射回原图坐标时计算错误。
- 解决方案：
 - 实现了标准的 Letterbox 预处理：在缩放图像时保持原始长宽比，使用灰色边框填充 (Padding) 不足的部分。
 - 实现了 坐标逆变换 (scale_coords)：在后处理时，先减去 Padding 偏移量，再除以缩放比例，精确地将坐标还原到原视频分辨率。

4.类别识别错误

- 问题：皮筏艇和渔船形状高度相似，加上角度等等特别容易识别错误
- 解决：由于默认识别方式是靠单帧独立，那么后续就很有可能错误。因此加入时序逻辑控制，简单来说就是多帧判断确认类别，不仅仅相信一帧。加权投票